

GODOT
Game engine

L'Eserciziario

Corso di Godot

Corso a cura del prof. Nicola Regge

Come funziona ogni esercizio

Ogni esercizio ha **4 livelli di aiuto**. Prova sempre da solo, e apri il livello successivo **solo se sei bloccato**:

1.  **Descrizione** — cosa devi ottenere.
2.  **Aiuto** — un indizio su come fare.
3.  **La scena** — quali nodi creare, i “mattoncini”.
4.  **Codice completo** — la soluzione da copiare/incollare.

*Regola: se copi il **Codice completo**, poi devi saper **spiegare a voce cosa fa, riga per riga**. Se lo sai spiegare, hai imparato lo stesso.*

Nel file `.md` i livelli 2–4 sono a scomparsa: clicca sul triangolino per aprirli. Nel PDF sono già aperti.

***Oltre agli esercizi numerati ci sono i “Progetti BOSS”:** giochi già pronti, più grossi, che non si copiano riga per riga — si **aprono, si giocano e si rendono propri**: cambi colori, titolo, ci metti una tua foto. Sono il premio: la cosa figa da mostrare subito agli amici. Il codice è già nel repository, lo capirai un pezzo alla volta.*

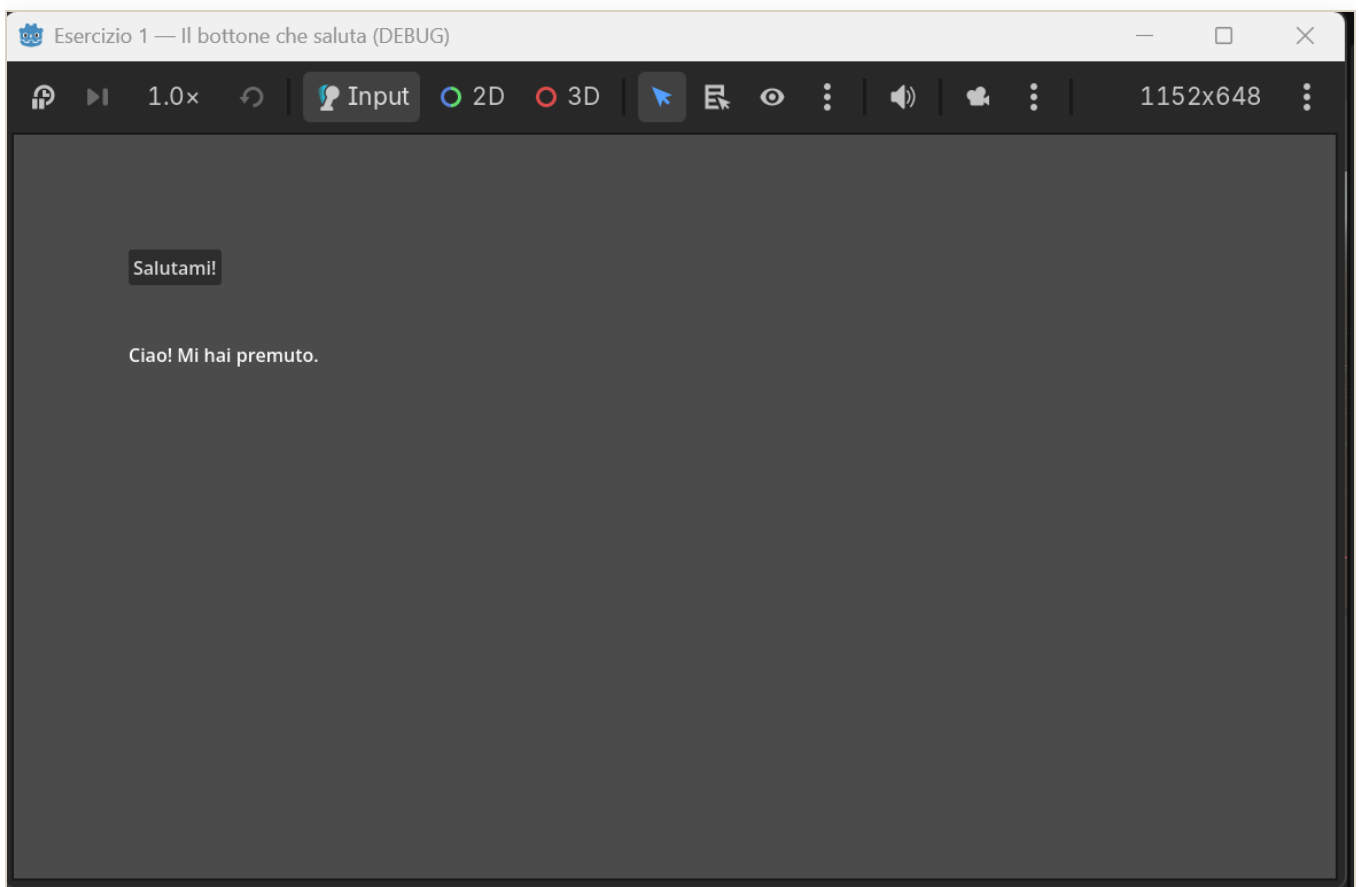
ESERCIZIO 1

Il bottone che saluta

Ponte da Lazarus: è il tuo `Button1Click` che cambia una `Caption`!

Descrizione

Crea una schermata con **un bottone** e **una scritta**. Quando premi il bottone, la scritta deve cambiare, per esempio da “...” a “Ciao! Mi hai premuto”.



Il bottone che saluta: quando lo premi, la scritta cambia.

Fallo tuo: scegli **tu** la frase del saluto e il **colore** della scritta — così il gioco è già tuo. Nessuno lo farà uguale al tuo!

Aiuto

- In Godot il bottone è il nodo **Button**, la scritta è il nodo **Label**.
- La proprietà `text` di Godot è come la **Caption** di Lazarus.

- L'evento "click" in Godot si chiama **segnale** `pressed`. Lo colleghi a una tua funzione con `bottone.pressed.connect(la_mia_funzione)`.

● La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio: **Button** → rinominalo `bottoneScena`.
3. Figlio: **Label** → rinominalo `etichettaScena`.
4. Attacca uno **script** al nodo radice `Main`.

● Codice completo

```
extends Node2D
```

```
# $NomeNodo = prende un nodo figlio per nome (come Button1 in Lazarus)
```

```
@onready var bottoneVar: Button = $bottoneScena
```

```
@onready var etichettaVar: Label = $etichettaScena
```

```
func _ready() -> void:
```

```
# Posizioniamo i due elementi così non si sovrappongono
```

```
bottoneVar.position = Vector2(100, 100)
```

```
bottoneVar.text = "Salutami!"
```

```
# <- come Button.Caption in Lazarus
```

```
etichettaVar.position = Vector2(100, 180)
```

```
etichettaVar.text = "..."
```

```
# FALLO TUO: scegli il colore della scritta, rosso verde blu da 0 a 1
```

```
etichettaVar.add_theme_color_override("font_color", Color(1, 0, 0)) # rosso
```

```
# Colleghiamo il "click" (segnale pressed) alla nostra funzione
```

```
bottoneVar.pressed.connect(_quando_premo)
```

```
# Questa e' come il tuo Button1Click di Lazarus
```

```
func _quando_premo() -> void:
```

```
# FALLO TUO: scrivi qui il TUO saluto
```

```
etichettaVar.text = "Ciao! Mi hai premuto."
```

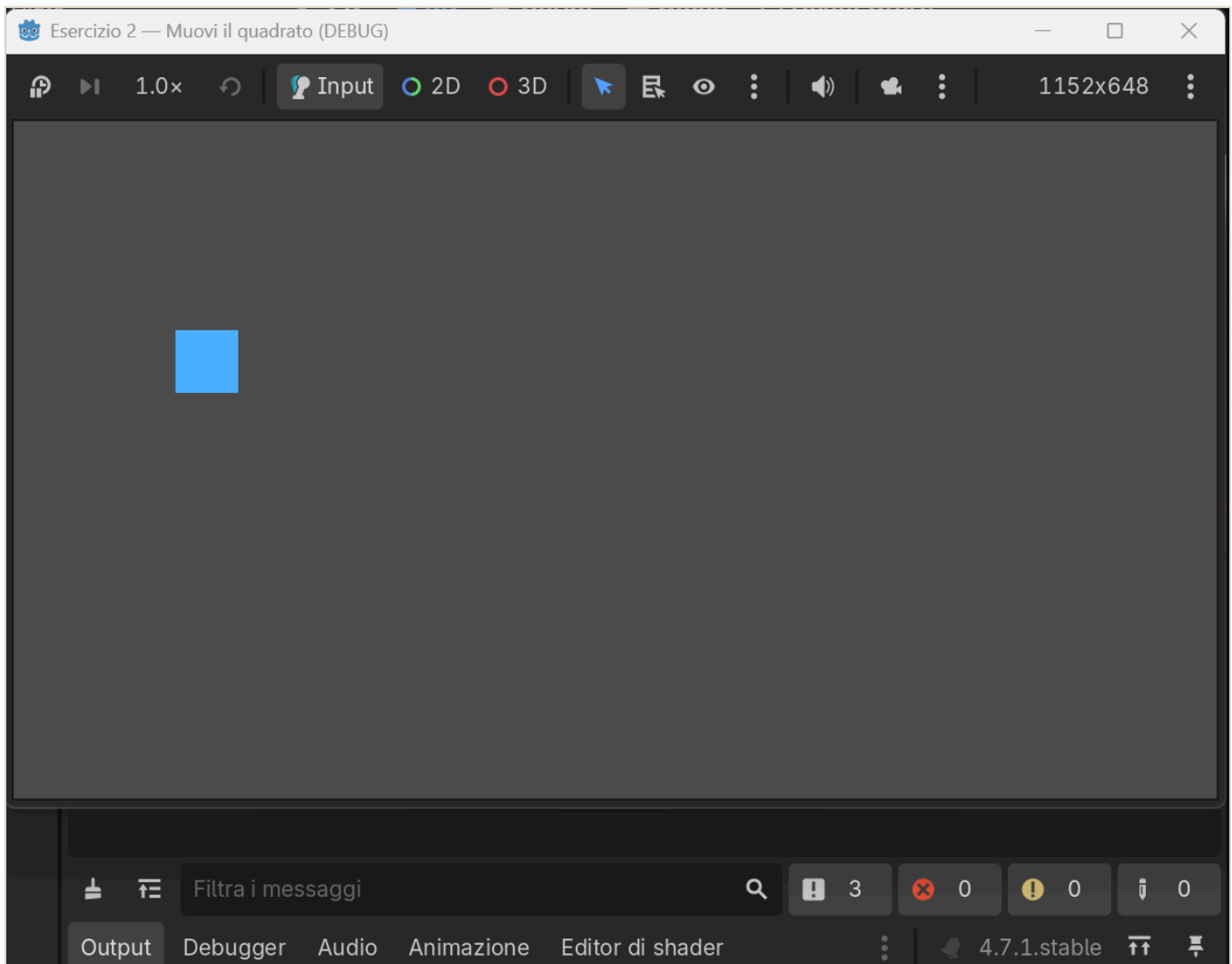
ESERCIZIO 2

Muovi il quadrato

Concetto nuovo: il game loop, cioè' `_process`.

Descrizione

Fai comparire un **quadrato** che puoi muovere in tutte le direzioni con le **freccie** della tastiera.



Il quadrato azzurro che si muove con le frecce.

Aiuto

- Un quadrato colorato semplice = nodo **ColorRect**.

- Il movimento va scritto in `_process(delta)`: e' la funzione che gira ~60 volte al secondo. In Lazarus non c'era: il programma stava fermo.
- Le frecce si leggono con `Input.is_action_pressed("ui_left")`, e allo stesso modo `ui_right`, `ui_up`, `ui_down`.
- Moltiplica sempre la velocita' per `delta`, cosi' va uguale su ogni PC.

● La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio: **ColorRect** → rinominalo `quadratoScena`.
3. Attacca uno **script** al nodo radice `Main`.

● Codice completo

```
extends Node2D

@onready var quadratoVar: ColorRect = $quadratoScena
const VELOCITA: float = 300.0 # pixel al secondo

func _ready() -> void:
    quadratoVar.size = Vector2(60, 60)
    quadratoVar.color = Color(0.3, 0.7, 1.0) # azzurro
    quadratoVar.position = Vector2(200, 200)

# _process gira a OGNI fotogramma: qui muoviamo il quadrato
func _process(delta: float) -> void:
    if Input.is_action_pressed("ui_left"):
        quadratoVar.position.x -= VELOCITA * delta
    if Input.is_action_pressed("ui_right"):
        quadratoVar.position.x += VELOCITA * delta
    if Input.is_action_pressed("ui_up"):
        quadratoVar.position.y -= VELOCITA * delta
    if Input.is_action_pressed("ui_down"):
        quadratoVar.position.y += VELOCITA * delta
```

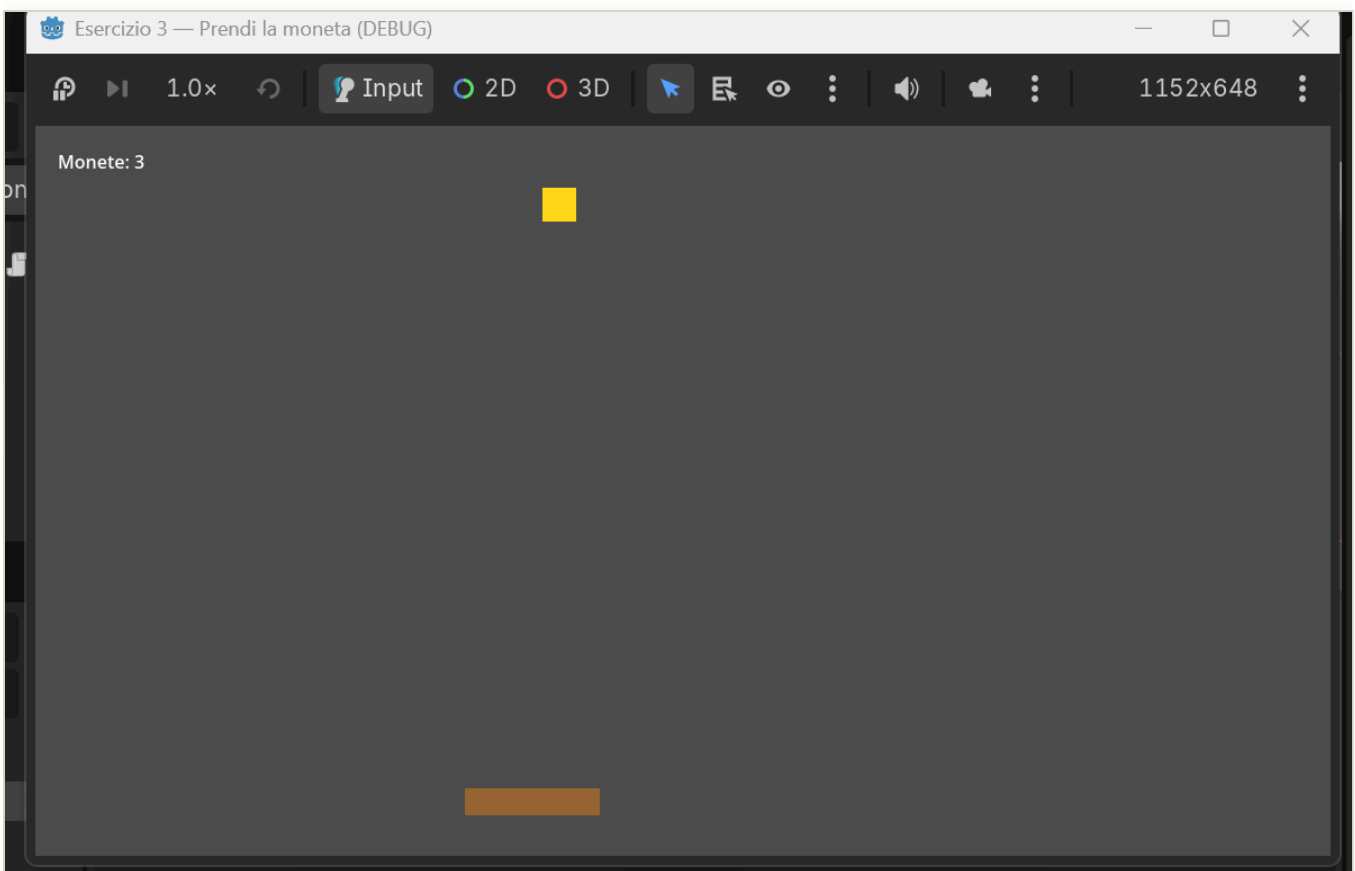
ESERCIZIO 3

Prendi la moneta

Mette insieme: movimento + oggetto che cade + punteggio. Verso il gioco vero.

Descrizione

Un **cestino** in basso, che muovi con le frecce $\leftarrow \rightarrow$, e una **moneta** che cade dall'alto. Se la prendi col cestino fai **+1 punto** e la moneta riparte dall'alto in una colonna a caso. Mostra il punteggio a schermo.



Prendi le monete col cestino: il punteggio sale.

Aiuto

- Cestino e moneta: due **ColorRect**. Il punteggio: un **Label**.
- Muovi il cestino in `_process`, come nell'Esercizio 2 ma solo sinistra/destra.
- Fai scendere la moneta ogni fotogramma: `moneta.position.y += velocita * delta`.
- Per capire se il cestino “tocca” la moneta usa i rettangoli: `Rect2(a.position, a.size).intersects(Rect2(b.position, b.size))`.

- Quando la moneta esce sotto o è presa, rimettila in alto a una ☐ a caso.

● La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio **ColorRect** → `cestinoScena`.
3. Figlio **ColorRect** → `monetaScena`.
4. Figlio **Label** → `punteggioScena`.
5. Attacca uno **script** al nodo radice `Main`.

● Codice completo

```
extends Node2D

@onready var cestinoVar: ColorRect = $cestinoScena
@onready var monetaVar: ColorRect = $monetaScena
@onready var punteggioVar: Label = $punteggioScena

const VELOCITA_CESTINO: float = 500.0
const VELOCITA_MONETA: float = 300.0

var punti: int = 0
var larghezza: float

func _ready() -> void:
    larghezza = get_viewport_rect().size.x
    # Cestino in basso
    cestinoVar.size = Vector2(120, 24)
    cestinoVar.color = Color(0.6, 0.4, 0.2)
    cestinoVar.position = Vector2(larghezza / 2.0 - 60, get_viewport_rect().size.y - 60)
    # Moneta
    monetaVar.size = Vector2(30, 30)
    monetaVar.color = Color(1.0, 0.85, 0.1)
    _rimetti_in_alto()
    # Punteggio
    punteggioVar.position = Vector2(20, 20)
    _aggiorna_punteggio()

func _process(delta: float) -> void:
    # Muovi il cestino
    if Input.is_action_pressed("ui_left"):
        cestinoVar.position.x -= VELOCITA_CESTINO * delta
    if Input.is_action_pressed("ui_right"):
        cestinoVar.position.x += VELOCITA_CESTINO * delta
```

```

    cestinoVar.position.x = clamp(cestinoVar.position.x, 0, larghezza -
cestinoVar.size.x)

    # Fai scendere la moneta
    monetaVar.position.y += VELOCITA_MONETA * delta

    # Presa?
    if Rect2(cestinoVar.position,
cestinoVar.size).intersects(Rect2(monetaVar.position, monetaVar.size)):
        punti += 1
        _aggiorna_punteggio()
        _rimetti_in_alto()
    # Persa (uscita sotto)?
    elif monetaVar.position.y > get_viewport_rect().size.y:
        _rimetti_in_alto()

func _rimetti_in_alto() -> void:
    var x := randf_range(0, larghezza - monetaVar.size.x)
    monetaVar.position = Vector2(x, -monetaVar.size.y)

func _aggiorna_punteggio() -> void:
    punteggioVar.text = "Monete: %d" % punti

```

Acchiappa le stelle

Concetto nuovo: le vite e il Game Over. Il gioco ora si può perdere e finire.

Descrizione

Una **navetta** in basso, che muovi con le frecce $\leftarrow \rightarrow$, e una **stella** che cade dall'alto. Se la prendi con la navetta fai +1. Se una stella tocca terra **perdi una vita**. Parti con **3 vite**: quando arrivi a zero è **GAME OVER** e premi **INVIO** per ricominciare. È l'Esercizio 3 che cresce.

SCREENSHOT DA INSERIRE

La navetta prende le stelle: se ne perdi tre è Game Over.

immagini/es4-gioca.png

Aiuto

- È l'Esercizio 3 con altri vestiti: la **navetta** è il cestino, la **stella** è la moneta. Il movimento e la “presa” funzionano allo stesso modo.
- La novità sono due variabili: `vite` (parte da 3) e `in_gioco` (vero/falso).
- Quando una stella esce sotto: `vite -= 1`. Se `vite <= 0`, chiama il Game Over.
- Nel Game Over metti `in_gioco = false` e mostra la Label “GAME OVER”.
- In `_process`, se **non** sei più in gioco, aspetta solo il tasto INVIO (`ui_accept`) per far ripartire tutto da capo.

La scena — i nodi da creare

1. Nodo radice: **Node2D**, rinominalo `Main`.
2. Figlio **ColorRect** → `navettaScena`.
3. Figlio **ColorRect** → `stellaScena`.
4. Figlio **Label** → `hudScena` (punteggio e vite).
5. Figlio **Label** → `gameoverScena` (la scritta di fine partita).

Codice completo

```
extends Node2D

@onready var navettaVar: ColorRect = $navettaScena
@onready var stellaVar: ColorRect = $stellaScena
@onready var hudVar: Label = $hudScena
@onready var gameoverVar: Label = $gameoverScena

const VELOCITA_NAVETTA: float = 500.0
const VELOCITA_STELLA: float = 300.0
const VITE_INIZIALI: int = 3

var punti: int = 0
var vite: int = VITE_INIZIALI
var in_gioco: bool = true
var larghezza: float

func _ready() -> void:
    larghezza = get_viewport_rect().size.x
    navettaVar.size = Vector2(90, 20)
    navettaVar.color = Color(0.3, 0.7, 1.0)
    navettaVar.position = Vector2(larghezza / 2.0 - 45, get_viewport_rect().size.y
    - 50)
    stellaVar.size = Vector2(28, 28)
    stellaVar.color = Color(1.0, 0.85, 0.2)
    hudVar.position = Vector2(20, 20)
    gameoverVar.position = Vector2(larghezza / 2.0 - 140,
    get_viewport_rect().size.y / 2.0 - 20)
    gameoverVar.text = "GAME OVER\nPremi INVIO per ricominciare"
    gameoverVar.visible = false
    _rimetti_in_alto()
    _aggiorna_hud()

func _process(delta: float) -> void:
    if not in_gioco:
        if Input.is_action_just_pressed("ui_accept"):
            _ricomincia()
            return
        if Input.is_action_pressed("ui_left"):
            navettaVar.position.x -= VELOCITA_NAVETTA * delta
        if Input.is_action_pressed("ui_right"):
            navettaVar.position.x += VELOCITA_NAVETTA * delta
            navettaVar.position.x = clamp(navettaVar.position.x, 0, larghezza -
            navettaVar.size.x)

    stellaVar.position.y += VELOCITA_STELLA * delta
```

```

    if Rect2(navettaVar.position,
navettaVar.size).intersects(Rect2(stellaVar.position, stellaVar.size)):
        punti += 1
        _aggiorna_hud()
        _rimetti_in_alto()
    elif stellaVar.position.y > get_viewport_rect().size.y:
        vite -= 1
        _aggiorna_hud()
        _rimetti_in_alto()
    if vite <= 0:
        _game_over()

func _rimetti_in_alto() -> void:
    var x := randf_range(0, larghezza - stellaVar.size.x)
    stellaVar.position = Vector2(x, -stellaVar.size.y)

func _game_over() -> void:
    in_gioco = false
    gameoverVar.visible = true

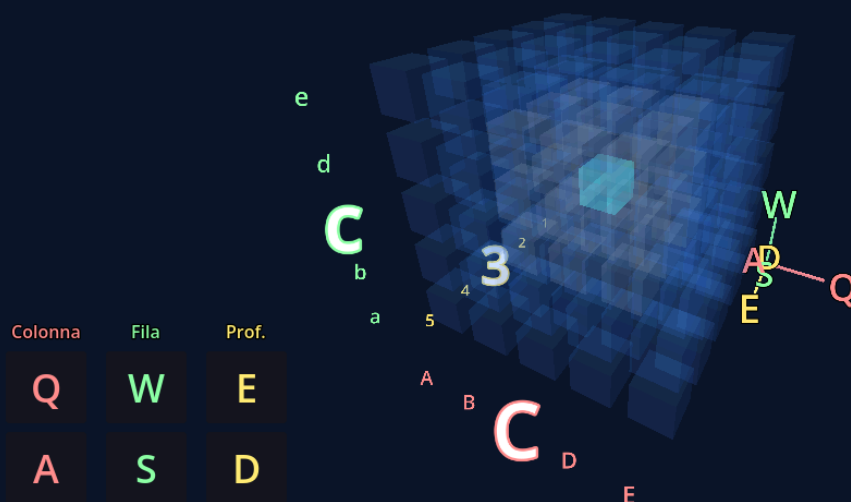
func _ricomincia() -> void:
    punti = 0
    vite = VITE_INIZIALI
    in_gioco = true
    gameoverVar.visible = false
    _rimetti_in_alto()
    _aggiorna_hud()

func _aggiorna_hud() -> void:
    hudVar.text = "Stelle: %d    Vite: %d" % [punti, vite]

```

Fallo tuo: cambia il colore e la dimensione della navetta e delle stelle, la velocità, o le vite di partenza. Stesso gioco, tema diverso: se al posto delle stelle metti gli organi che cadono dal tavolo, hai il **Chirurgo pasticcione** (nella cartella `chirurgo-pasticcione/`). Nella cartella `acchiappa-le-stelle/` trovi anche una versione più ricca, con tante stelle insieme, da studiare. Il primo “progetto boss”: una battaglia navale in 3D già giocabile. Si apre, si gioca, si rende proprio.

Mirino: C c 3 · Acqua... la zona 3x3x3 è esplosa. Continua! (bombe: 1)



Affonda la Bonomi in azione: il cubo d'acqua con il mirino verde, le coordinate scritte attorno al cubo e, in basso a sinistra, i comandi colorati dei tre assi (Colonna Q/A, Fila W/S, Profondità E/D).

Descrizione

La solita battaglia navale, ma **in tre dimensioni**: al posto della griglia a righe e colonne c'è un **cubo** di celle d'acqua. Dentro è nascosto **un sottomarino**. Muovi un **mirino** e lanci una **bomba di profondità**: esplode e colpisce una zona **3×3×3** attorno al punto. Se il sottomarino è lì → **COLPITO!** Questo gioco è **già fatto**: il tuo compito è **aprirlo, giocarci e farlo tuo**.

Aiuto — come aprirlo e giocarci

1. [APP – Godot] finestra iniziale, il Gestore progetti, in alto a destra **Importa** → scegli la cartella **battaglia-navale-3d** e il file **project.godot** → **Importa e modifica**.
2. Premi **F5** per eseguire. All'avvio scegli **quanti cubi per lato**, da **4** facilissimo, fino a **10** difficile.
3. Comandi, con le lettere disposte come **tre colonne della tastiera**:
4. **Q / A** = Colonna, in rosso · **W / S** = Fila, in verde · **E / D** = Profondità, in giallo
5. **SHIFT + le stesse lettere** = gira il cubo · **dito/mouse trascinato** = gira il cubo
6. **SPAZIO** = lancia la bomba · **↻ / INVIO** = rigioca e richiede di nuovo la difficoltà

● Fallo tuo — la parte più importante

Apri `battaglia-navale-3d/main.gd` e cambia queste cose per rendere il gioco **tuo**. Dopo ogni modifica premi `F5` e guarda l'effetto:

- **I colori dell'acqua e del mirino:** in alto trovi righe tipo `const COL_ACQUA := Color(...)` e `const COL_MIRINO := Color(...)`. Cambia i tre numeri, rosso verde blu da 0 a 1, e avrai il **tuo stile**.
- **La tua foto al posto di Serena:** metti un file `serena.jpg`, una tua foto o un meme) nella cartella `battaglia-navale-3d/`: comparirà quando affondi il sottomarino, lampeggiando con il teschio dei pirati.
- **Il titolo del gioco:** in `project.godot`, alla voce `config/name="..."`, scrivi il **nome che vuoi tu**.
- **La difficoltà di partenza / la potenza della bomba:** prova a cambiare `RAGGIO_BOMBA`: 1 è la zona $3 \times 3 \times 3$, 2 è la zona $5 \times 5 \times 5$, molto più potente.

Mostralo: quando l'hai personalizzato, fai una partita davanti a un compagno. “Questo l'ho fatto **io**” vale più di qualsiasi voto.

● Il codice completo — dov'è e com'è fatto

Il codice **c'è già tutto** ed è versionato nel repository, nel file `battaglia-navale-3d/main.gd`, circa 600 righe. Non va copiato a mano: è il nostro **progetto boss**, lo leggeremo **un pezzo alla volta**.

Le idee sono le **stesse degli esercizi precedenti**, portate in 3D:

- il **game loop** `_process(delta)` per girare il cubo, come nell'Esercizio 2;
- **leggere i tasti** con `Input.is_key_pressed(...)`, come nel muovere il quadrato;
- **costruire tutto da codice:** celle, luci, telecamera, bottoni, invece che a mano.

Regola d'oro, valida anche qui: se sai **spiegare a voce** cosa fa un pezzo di codice, quel pezzo è tuo. Partiremo dai pezzi più facili, i colori e i comandi, e saliremo piano piano.